

S2S-09: Step 9 — Optimize: Cutover Validation and Decommission

Series: S2S | **Notebook:** 9 of 9 | **Phase:** Run | **Step:** Optimize | **Created:** March 2026 | **Last Updated:** 04/04/2026

Overview

This is the final step of the SaaS-to-SaaS migration. With parallel operation validated (Step 8), Davis AI baselines established, and stakeholders trained, you are ready to execute the cutover. This notebook covers the go/no-go decision, cutover execution steps, post-cutover validation, rollback procedures, source tenant decommission, and a lessons learned framework.

S2S Migration Journey — 3 Phases / 9 Steps

Plan: 1. Discover | 2. Strategize | 3. Design

Upgrade: 4. Prepare | 5. Execute | 6. Integrate

Run: 7. Expand | 8. Enable | **9. Optimize**

Order of Operations (Steps 9–11)

Within the 9-step framework, Step 9 maps to three operational actions:

Operation	Action	Description
9	Validate	Final data flow and performance validation
10	Cutover	Full switch to target tenant
11	Decommission	Source tenant deactivation and deprovisioning

Table of Contents

1. [Go/No-Go Checklist](#)
 2. [Cutover Execution Steps](#)
 3. [Post-Cutover Validation Queries](#)
 4. [Rollback Procedures](#)
 5. [Source Tenant Decommission](#)
 6. [Documentation and Handover](#)
 7. [Lessons Learned Framework](#)
 8. [Series Conclusion](#)
 9. [Step Completion Checklist](#)
-

Prerequisites

Requirement	Details
Step 8 Complete	Parallel operation validated, stakeholders trained, Davis baselines established
Go/No-Go Authority	Decision-maker identified and available for cutover approval
Maintenance Window	Scheduled and communicated to all stakeholders
Rollback Plan	Documented and tested — source tenant still active and receiving data
Communication Ready	Cutover announcement drafted for all stakeholder audiences

1. Go/No-Go Checklist

The go/no-go decision should be made in a structured meeting with all stakeholders present. Every item below must be **Go** for cutover to proceed. Any **No-Go** item requires documented remediation and a reschedule.

Infrastructure Readiness

Checkpoint	Status	Verified By
All hosts reporting to target tenant	☐	Platform team
All K8s clusters monitored via DynaKube in target	☐	Platform team
Cloud integrations (AWS/Azure/GCP) active in target	☐	Cloud team
ActiveGates routing to target tenant	☐	Platform team

Configuration Readiness

Checkpoint	Status	Verified By
All Settings 2.0 configuration deployed	☐	Platform team
OpenPipeline rules validated (enrichment, routing, masking)	☐	Platform team
SLO definitions deployed and evaluating	☐	SRE team
Alerting profiles and notification rules active	☐	SRE team
Dashboards and notebooks accessible	☐	All teams
Workflows and automations tested	☐	Platform team

Security Readiness

Checkpoint	Status	Verified By
IAM policies deployed (groups, bindings)	☐	Security team
SSO/SAML configured and tested	☐	Identity team
API tokens generated and distributed	☐	Platform team
Credential vault populated	☐	Security team

Data Readiness

Checkpoint	Status	Verified By
Log volumes match between source and target ($\pm 10\%$)	[]	Platform team
Span volumes match between source and target ($\pm 10\%$)	[]	Platform team
Davis AI baselines stable (2+ weeks of data)	[]	SRE team
SLOs evaluating within expected range	[]	SRE team

2. Cutover Execution Steps

Execute these steps in order during the scheduled maintenance window. Each step includes estimated duration and rollback instructions.

Step	Action	Duration	Rollback
1	Announce — Send cutover start notification to all stakeholders	5 min	N/A
2	Suppress — Enable maintenance window in source tenant	5 min	Disable maintenance window
3	Redirect — Update DNS/proxy for API endpoints to target tenant	15 min	Revert DNS/proxy
4	Verify — Confirm agents are reporting to target tenant only	15 min	Re-enable dual reporting
5	Enable — Activate alerting in target tenant (remove staging routing)	10 min	Re-route to staging
6	Validate — Run post-cutover validation queries (Section 3)	30 min	If validation fails, rollback
7	Disable — Stop data collection in source tenant (set to read-only)	10 min	Re-enable collection
8	Communicate — Send cutover complete notification	5 min	N/A

Step	Action	Duration	Rollback
9	Monitor — Active monitoring for 4–24 hours post-cutover	Ongoing	Full rollback if critical issues

Total estimated cutover window: 2–4 hours (including monitoring buffer)

Cutover Timing Recommendations

Timing	Pros	Cons
Weekend morning (low traffic)	Minimal user impact, low alert volume	Less staff available
Mid-week morning	Full team available	Higher traffic, more visible
End of business Friday	Weekend to stabilize	Fatigue risk, limited weekend support

3. Post-Cutover Validation Queries

Run these queries in the **target tenant** immediately after cutover to confirm that all data sources are flowing correctly.

Host Count Validation

```
// Post-cutover: Total host count by cloud provider
fetch dt.entity.host
| fieldsAdd provider = if(isNotNull(awsNameTag), then: "AWS",
    else: if(isNotNull(azureResourceGroupName), then: "Azure", else: "On-
Premises"))
| summarize count = count(), by:{provider}
| sort count desc

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes HOST
// | fieldsAdd provider = if(isNotNull(awsNameTag), then: "AWS",
// else: if(isNotNull(azureResourceGroupName), then: "Azure", else: "On-
// Premises"))
// | summarize count = count(), by:{provider}
// | sort count desc
```

Service Count Validation

```
// Post-cutover: Service inventory by technology
fetch dt.entity.service
| summarize count = count(), by:{serviceType}
| sort count desc

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes SERVICE
// | summarize count = count(), by:{serviceType}
// | sort count desc
```

Log Flow Validation

```
// Post-cutover: Log volume by loglevel (last 1 hour)
// Compare with pre-cutover baseline from source tenant
fetch logs, from:-1h
| summarize record_count = count(), by:{loglevel}
| sort record_count desc
```

Span Flow Validation

```
// Post-cutover: Span volume by span.kind (last 1 hour)
fetch spans, from:-1h
| summarize span_count = count(), by:{span.kind}
| sort span_count desc
```

Enrichment Validation

```
// Post-cutover: Verify OpenPipeline enrichment is active
fetch logs, from:-1h
| fieldsAdd has_security_context = isNotNull(dt.security_context)
| summarize
    total = count(),
    enriched = countIf(has_security_context == true)
| fieldsAdd enrichment_pct = round(toDouble(enriched) / toDouble(total) *
100.0, decimals: 1)
| fieldsAdd status = if(enrichment_pct > 50.0, then: "Enrichment healthy",
else: "Check OpenPipeline config")
```

Davis Problem Validation

```
// Post-cutover: Active Davis problems (should be normal operational level)
fetch dt.davis.problems, from:-24h
| filter event.status == "ACTIVE"
| summarize active_problems = count()
| fieldsAdd assessment = if(active_problems < 10, then: "Normal range",
    else: if(active_problems < 50, then: "Elevated – review problems",
else: "High – investigate immediately"))
```

MTTR Baseline

```
// Post-cutover: Mean time to resolve (MTTR) for closed problems
// This establishes the MTTR baseline in the target tenant
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| filter dt.davis.is_frequent_event == false and dt.davis.is_duplicate == false
| fieldsAdd duration_hours = toLong(resolved_problem_duration) / 3600000000000.0
| summarize
    avg_mttr_hours = avg(duration_hours),
    median_mttr_hours = median(duration_hours),
    problems_resolved = count()
| fieldsAdd avg_mttr_hours = round(avg_mttr_hours, decimals: 2),
    median_mttr_hours = round(median_mttr_hours, decimals: 2)
```

4. Rollback Procedures

If critical issues are discovered during or after cutover, follow these rollback procedures by component.

Rollback Decision Criteria

Severity	Criteria	Action
Critical	>25% of hosts not reporting, or no log/span flow	Full rollback immediately
High	Key SLOs evaluating at 0%, or critical alerts not firing	Partial rollback, keep agents dual-reporting
Medium	Minor data gaps, non-critical dashboards missing data	Continue with remediation plan
Low	Cosmetic issues, minor configuration differences	Fix forward

Rollback by Component

Component	Rollback Action	Time to Rollback
OneAgent	Update <code>host_group</code> or connection endpoint back to source tenant	5–15 min per host (automated via deployment tool)
DynaKube	Update DynaKube CR <code>apiUrl</code> to source tenant	5 min + pod restart
Cloud integrations	Re-enable in source tenant, disable in target	15 min
Alerting	Disable target alerting, re-enable source	10 min
SLOs	No rollback needed — source SLOs still evaluating during parallel	N/A
DNS/proxy	Revert API endpoint routing to source tenant	15 min

5. Source Tenant Decommission

After a successful cutover and monitoring buffer period, the source tenant can be decommissioned. This is a gradual process — do not rush it.

Decommission Timeline

Week	Action	Notes
Week 0	Cutover complete — source set to read-only	No new data ingested
Week 1–2	Active monitoring buffer — source available for historical queries	Keep source accessible
Week 3–4	Export any remaining artifacts (problem reports, session replay screenshots)	Document what was exported
Week 5–6	Disable source tenant monitoring agents (remove OneAgents, delete DynaKube CRs)	Confirm no data flowing to source

Week	Action	Notes
Week 7	Revoke all API tokens and OAuth clients in source tenant	Security cleanup
Week 8	Request tenant deprovisioning from Dynatrace account team	Final step

Pre-Decommission Checklist

Item	Action	Status
Historical problem reports exported	Export as PDF or CSV for compliance records	[]
Key dashboards screenshotted	Capture for reference	[]
Audit log exported	Required for compliance in some industries	[]
All users notified	Source tenant access will be revoked	[]
API tokens revoked	Remove all active tokens	[]
OAuth clients deleted	Remove all client credentials	[]
SSO/SAML disconnected	Remove identity provider integration	[]
Account team notified	Request formal deprovisioning	[]

6. Documentation and Handover

After migration, update all documentation to reflect the new environment.

Documents to Update

Document	Updates Required
Architecture diagrams	New tenant URLs, updated network flows
Runbooks	API endpoints, token references, escalation paths
Disaster recovery plan	New tenant in DR procedures

Document	Updates Required
Change management records	Migration completion, lessons learned
Vendor management	Updated contract references, licensing terms
Security documentation	New IAM architecture, token inventory
Onboarding guides	New tenant URLs, access request procedures

7. Lessons Learned Framework

Capture lessons learned while the migration is still fresh. Organize by category for maximum reusability.

Capture Template

Category	What Went Well	What Could Improve	Action Items
Planning	—	—	—
Configuration	—	—	—
Agent Migration	—	—	—
Data Validation	—	—	—
Stakeholder Communication	—	—	—
Cutover Execution	—	—	—
Tooling (Monaco/Terraform)	—	—	—
Davis AI / Baselines	—	—	—

Common Lessons from S2S Migrations

Lesson	Detail
Entity ID remapping takes longer than expected	Audit dashboards and SLOs before migration, not during

Lesson	Detail
Davis baselines need at least 2 full weeks	Plan parallel operation accordingly
Credential recreation is a bottleneck	Involve cloud and security teams early
Stakeholder fatigue is real	Keep communication concise and predictable
Extensions 2.0 must be reinstalled from Hub	Neither Monaco nor Terraform can export them
Webhook URLs are easily missed	Audit all notification integrations systematically

8. Series Conclusion

This completes the 9-step SaaS-to-SaaS migration framework.

Journey Summary

Phase	Step	Notebook	Focus
Plan	1	S2S-01	Discover — Inventory source tenant, identify migration scenario
Plan	2	S2S-02	Strategize — Select approach, timeline, risk assessment
Plan	3	S2S-03	Design — Target tenant architecture (IAM, buckets, pipelines)
Upgrade	4	S2S-04	Prepare — Build target tenant, deploy configuration
Upgrade	5	S2S-05	Execute — Redirect agents, migrate OneAgent and DynaKube
Upgrade	6	S2S-06	Integrate — Reconnect cloud integrations and extensions
Run	7	S2S-07	Expand — OpenPipeline, SLOs, alerting migration
Run	8	S2S-08	Enable — Parallel operation, stakeholder handover
Run	9	S2S-09	Optimize — Cutover, validation, decommission

Key Principles Reinforced Throughout

Principle	Why It Matters
The 90/10 Rule	90% of config migrates automatically; 10% requires 90% of the effort
Tags over entity IDs	Tag-based selectors survive migration; hardcoded entity IDs do not
Monaco for config, Terraform for IAM	Use each tool where it excels
Parallel operation is mandatory	Historical data does not migrate — plan for dual-tenant operation
Davis AI needs time	Baselines require 2–4 weeks; communicate this to stakeholders
Communicate early and often	Migration success is as much about people as technology

9. Step Completion Checklist

Checkpoint	Status
Go/no-go meeting completed — all four categories green	[]
Cutover executed during scheduled maintenance window	[]
Post-cutover validation queries all pass (hosts, services, logs, spans, enrichment)	[]
Davis problem volume within normal range	[]
MTTR baseline established in target tenant	[]
Rollback plan confirmed not needed (or rollback executed and re-cutover scheduled)	[]
Source tenant set to read-only	[]
Source tenant decommission timeline agreed	[]
All documentation updated (runbooks, architecture, DR, security)	[]

Checkpoint	Status
Lessons learned captured and shared	[]
Migration formally closed with stakeholder sign-off	[]

Summary

In Step 9 — the final step — you:

- Conducted a structured go/no-go assessment across infrastructure, configuration, security, and data readiness
- Executed the cutover following a 9-step sequence with defined rollback procedures for each step
- Validated the target tenant with DQL queries confirming host counts, service counts, log flow, span flow, and enrichment
- Established MTTR and Davis problem baselines in the target tenant
- Planned source tenant decommission on an 8-week timeline
- Captured lessons learned for future migrations

The SaaS-to-SaaS migration is complete. The target tenant is now the production monitoring environment.

Additional Resources

- [Dynatrace Grail Data Lakehouse](#)
- [Davis AI Anomaly Detection](#)
- [Monaco Configuration as Code](#)
- [Dynatrace Terraform Provider](#)
- [DQL Reference](#)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.